# GitPress - User Manual

> ***Version:*** *0.9.0* ***License:*** *MIT* ***Platforms:*** *Windows, macOS, Linux (DEB/RPM)*

GitPress is a cross-platform Git GUI desktop app inspired by Git Extensions, built with Electron, React, and TypeScript. It provides a modern interface with Catppuccin theming for managing Git repositories visually and comprehensively.

---

## Table of Contents


1. Installation and Requirements
2. First Launch
3. Main Interface
    - Menu Bar
    - Toolbar
    - Sidebar
    - Commit Graph
    - Commit Details Panel
    - Status Bar
    - Conflict Banner
4. Repository Management
    - Create a Repository
    - Open a Repository
    - Clone a Repository
    - Scan the Filesystem
    - Favorite and Recent Repositories
5. Commit and Staging
    - Commit Dialog
    - Staging and Unstaging Files
    - Line and Hunk Staging
    - Amend Last Commit
    - Commit Templates
    - AI Commit Messages
6. Branches
    - Create a Branch
    - Checkout a Branch
    - Rename and Delete Branches
    - Delete Remote Branches
    - Set Upstream Branch
7. Merge
    - Merge Dialog
    - Merge Options
    - Abort and Continue
8. Rebase
    - Standard Rebase
    - Interactive Rebase
    - Conflict Handling During Rebase
9. Cherry-Pick
10. Revert

11. Squash
12. Conflict Resolution
    - Built-in 3-Way Editor
    - External Merge Tool
    - AI-Assisted Resolution
13. Diff and File Viewing
    - Diff Viewer
    - File History
    - Blame View
    - Commit Comparison
    - File Context Menu
14. Tags
15. Stash
16. Remotes
    - Fetch, Pull, and Push
    - Remote Management
    - Stale Remote Branches
17. Reset
18. Reflog
19. Bisect
20. Worktrees
21. Submodules
22. Git LFS (Large File Storage)
23. Patches
24. Archive and Export
25. Git Notes
26. Pull Request Integration
27. Changelog
28. Integrated Console
29. Command Log
30. Statistics
    - Author Statistics
    - Codebase Statistics
31. Git Accounts
32. Settings
    - General
    - Accounts
    - Git Config
    - Fetch
    - Commit
    - Diff and Graph
    - Merge Tool
    - Advanced
    - AI / MCP
33. AI and MCP Integration
    - Supported AI Providers
    - MCP Server
    - AI Features
34. Automatic Updates

35. Keyboard Shortcuts
36. Themes
37. Customizable Layout
38. Internationalization

---

# 1. Installation and Requirements

## System Requirements

- **Node.js** 20 or higher (only for building from source)
- **Git** 2.30 or higher (must be in the system PATH)
- **Operating System:** Windows 10+, macOS (Apple Silicon), Linux (Debian/RPM-based distributions)

## Install from Release

Download the appropriate installer from the GitHub Releases page:

- **Windows:** MSI installer ( `.msi` ) or Portable ZIP archive
- **macOS:** ZIP archive ( `.zip` ) for Apple Silicon
- **Linux:** `.deb` package (Debian/Ubuntu) or `.rpm` package (Fedora/RHEL)

## Security Warnings on First Launch

GitPress is currently distributed **without code signing certificates**. This means your operating system may show security warnings when you first run the application. These warnings are expected and can be safely bypassed.

### Windows (SmartScreen)

When launching the installer or the app for the first time, Windows SmartScreen may show a warning saying **"Windows protected your PC"**:

1. Click **"More info"**
2. Click **"Run anyway"**

This warning will not appear again after the first launch. If you downloaded the Portable ZIP, you may also need to right-click the `.exe` file, select **Properties**, and check **"Unblock"** before running it.

### macOS (Gatekeeper)

macOS Gatekeeper will block the app because it is from an unidentified developer:

**Method 1 — Right-click to open (recommended):**

1. Locate `GitPress.app` in Finder
2. **Right-click** (or Control-click) the app and select **"Open"**
3. In the dialog that appears, click **"Open"** again

**Method 2 — Remove quarantine attribute:**

```
xattr -cr /Applications/GitPress.app
```

**Method 3 — Allow in System Preferences:**

1. Try to open the app normally (it will be blocked)
2. Go to **System Settings > Privacy & Security**
3. Scroll down — you will see a message about GitPress being blocked

4. Click **"Open Anyway"**

### Build from Source

```
git clone https://github.com/CODEPRESS-SRL/git-press.git
cd git-press
npm install
npm start          # Launch in development mode
npm run make       # Create distributable packages
```

---

## 2. First Launch

On first launch, GitPress shows a **welcome screen** with:

- **Quick actions:** Create, Open, Clone, or Scan for repositories
- **Recent repositories:** List of recently opened repositories (initially empty)
- **Shortcut hints:** `Ctrl+O` to open, `Ctrl+N` to create

The application remembers the last opened repository and automatically reopens it on subsequent sessions.

---

## 3. Main Interface

The interface is divided into several areas, all customizable through the dockable panel system (dockview).

### 3.1 Menu Bar

The top menu bar provides four main menus:

**Start Menu**

| Item | Shortcut | Description |
|---|---|---|
| Create new repository... | | Initialize a new Git repository |
| Open repository... | `Ctrl+O` | Open an existing repository |
| Favorite repositories | | Submenu with repositories organized by category |
| Recent repositories | | Last 10 opened repositories |
| Clone repository... | | Clone a remote repository |
| Scan for repositories... | | Search for Git repositories in the filesystem |
| Exit | `Ctrl+Q` | Close the application |

**Dashboard Menu**

| Item | Shortcut | Description |
|---|---|---|
| Refresh | `F5` | Refresh the graph and status |
| Reset layout | | Restore the panel layout to default |

**Tools Menu**

| Item | Shortcut | Description |
|---|---|---|
| Git bash | `Ctrl+G` | Open a terminal in the repository directory |
| Stale remote branches... | | Find and delete obsolete remote branches |
| Settings... | `Ctrl+,` | Open settings |

**Help Menu**

| Item | Description |
|---|---|
| User manual | Open the online documentation (GitHub wiki) |
| Report an issue | Open the bug report page on GitHub |
| Check for updates... | Check for available updates |
| About GitPress | Show version, license, and information |

### 3.2 Toolbar

The toolbar appears below the menu bar and adapts to the context:

- **Without an open repository:** Shows only the Open and Init buttons
- **With an open repository:** Shows the full set of operations

Available buttons (left to right):

| Button | Description |
|---|---|
| Open | Open a repository |
| Init | Create a new repository |
| Refresh | Refresh graph and status |
| Account | Active Git account selector |
| Fetch | Download updates from remote (with dropdown for options) |
| Pull | Pull from remote (with dropdown: merge, rebase) |
| Push | Push to remote |
| Commit (`Ctrl+K`) | Open the Commit Dialog |
| Merge | Open the Merge Dialog |
| Rebase | Open the Rebase Dialog |
| Cherry-pick | Cherry-pick a commit |

The Fetch and Pull buttons have additional dropdowns with advanced options (fetch all, fetch prune, pull with rebase, pull with merge).

### 3.3 Sidebar

The left sidebar shows the repository tree with expandable sections:

- **Search bar:** Filter branches, remotes, and tags by name
- **Branches:** Local and remote branches with ahead/behind indicators
- **Remotes:** List of configured remotes
- **Tags:** All repository tags
- **Submodules:** Git submodules
- **Stashes:** List of saved stashes

Each section supports a **context menu** (right-click) with specific operations:

**Branch (right-click):**

- Checkout, Merge, Rebase, Delete, Rename, Set upstream

**Remote (right-click):**

- Add, Remove, Edit URL

**Tag (right-click):**

- Delete, Push to remote

**Stash (right-click):**

- Apply, Pop, Drop, Inspect

### 3.4 Commit Graph

The central panel shows the commit graph with:

- **Colored lines** for each branch/lane
- **Dots** for each commit (the HEAD commit has a larger dot with a white border and bold text)
- **Gravatar avatars** next to the author's name
- **Decorations:** Labels for branches and tags on each commit
- **Virtualized scrolling** to handle repositories with thousands of commits
- **Pagination:** "Load More" button to load additional commits (blocks of 500)

**Graph Filters**

Two filters are available in the graph toolbar:

1. **Branch name filter:** Text field to filter commits by branch name (substring match)
2. **Branch visibility filter:** Dropdown to include/exclude specific branches from the view

These filters are **saved per repository** and automatically restored when reopened.

**Commit Search**

Press `Ctrl+F` to open the search bar in the graph. You can search by:

- Commit message
- Author name
- Commit hash
- Reference name (branch/tag)

**Graph Context Menu (right-click on a commit)**

| Item | Description |
| --- | --- |
| Checkout | Checkout the commit or branch |
| Create branch | Create a new branch from this commit |
| Delete branch | Delete the branch pointing to this commit |
| Rename branch | Rename the branch |
| Merge into current | Merge this branch into the current branch |
| Rebase current onto | Rebase the current branch onto this commit |
| Cherry-pick | Apply this commit to the current branch |
| Revert | Revert this commit |
| Squash commits | Squash commits up to this point |
| Reset current branch | Reset the current branch to this commit |
| Create tag | Create a tag on this commit |
| Delete tag | Delete a tag |
| Delete remote branch | Delete the remote branch |
| Compare with HEAD | Compare this commit with HEAD |
| Compare with selected | Compare with another selected commit |
| Generate changelog | Generate a changelog from this commit |
| AI code review | AI-powered code review for this commit |

### 3.5 Commit Details Panel

When a commit is selected in the graph, the details panel shows two tabs:

#### Diff Tab

Shows the unified diff of the commit with:

- **Line-by-line** or **side-by-side** mode (toggleable)
- Syntax highlighting
- Colored lines (red = removed, green = added)

#### Files Tab

Shows the tree of modified files with:

- Status badges for each file (A = added, M = modified, D = deleted, R = renamed, C = copied)
- Click a file to view its diff
- Right-click for the file context menu

If no commit is selected, the panel shows HEAD information as a fallback.

### 3.6 Status Bar

The bottom bar shows:

- **Current branch** (e.g., `main`)
- **Change count:** Number of staged, unstaged, and untracked files
- **HEAD hash:** First 8 characters of the current commit hash
- **Accent color:** Changes based on the repository state (clean/dirty)

### 3.7 Conflict Banner

When a merge, rebase, or cherry-pick operation is in progress and there are conflicts, a banner appears at the top of the interface:

- **Red color:** There are unresolved conflicts
- **Green color:** All conflicts have been resolved
- **Progress:** For rebase, shows the current step (e.g., "Step 3/7")
- **Conflict counter:** Shows how many conflicts have been resolved (e.g., "2/5 resolved")

Available buttons:

| Button | Description |
|---|---|
| Resolve Conflicts | Open the conflict resolution editor |
| Skip Commit | Skip the current commit (during rebase) |
| Abort | Cancel the entire operation |
| Continue | Proceed with the operation after resolution |

---

## 4. Repository Management

### 4.1 Create a Repository

1. Click **Start > Create new repository...** or press the **Init** button in the toolbar
2. Select the directory where you want to create the repository
3. GitPress will run `git init` in the chosen directory
4. The new repository will be opened automatically

### 4.2 Open a Repository

- Shortcut: `Ctrl+O`
- Menu: **Start > Open repository...**
- Toolbar: **Open** button

A dialog will open to select the repository directory. The repository will be automatically added to the recent list.

### 4.3 Clone a Repository

Menu: **Start > Clone repository...**

The Clone Dialog provides the following options:

| Option | Description |
|---|---|

| | |
|---|---|
| URL | Remote repository address (HTTPS or SSH) |
| Destination | Local directory to clone into (with Browse button) |
| Branch | Specific branch to clone (optional) |
| Bare clone | Clone only the Git database without a working tree |
| Fetch submodules | Also download submodules |
| Shallow clone | Shallow clone with configurable depth |

### 4.4 Scan the Filesystem

Menu: **Start > Scan for repositories...**

The Scan function recursively searches for Git repositories in the filesystem:

1. Select the **root directory** to start searching from
2. Set the **maximum depth** (1-10, default 4)
3. Start the scan
4. Found repositories are displayed in real-time with a progressive count
5. All found repositories are automatically added to the recent list

### 4.5 Favorite and Recent Repositories

**Recent Repositories**

- Accessible from **Start > Recent repositories**
- Shows the last 10 opened repositories
- Repositories removed from the filesystem are automatically cleaned from the list

**Favorite Repositories**

- Accessible from **Start > Favorite repositories**
- Organized in customizable **categories**
- To add a repository to favorites: assign a category to the repository
- Categories can be created, renamed, and deleted

---

## 5. Commit and Staging

### 5.1 Commit Dialog

Open with: `Ctrl+K` or the **Commit** button in the toolbar.

The Commit Dialog is inspired by Git Extensions and features:

- **Upper-left panel:** List of unstaged files with status badges
- **Lower-left panel:** List of staged files with status badges
- **Center/right panel:** Diff view of the selected file
- **Message area:** Field for the commit message
- **Top bar:** Current branch and selected file path

### 5.2 Staging and Unstaging Files

In the Commit Dialog:

- **Stage:** Select files in the unstaged list and click the down arrow button (or double-click)
- **Unstage:** Select files in the staged list and click the up arrow button (or double-click)
- **Stage All / Unstage All:** Buttons to move all files at once
- **Discard:** Discard changes to selected files

### 5.3 Line and Hunk Staging

For granular control, you can stage individual lines or code blocks (hunks):

1. Select a file in the unstaged list
2. In the diff viewer, select the specific lines you want to stage
3. Use the **Stage Lines** or **Unstage Lines** buttons to operate on individual lines

> ***Note:*** *Line staging is not available for conflicted files.*

### 5.4 Amend Last Commit

In the Commit Dialog, the commit button has a **dropdown menu** with the **Amend** option:

- Modify the message of the last commit
- Add staged files to the last commit
- The message field is pre-filled with the last commit's message

### 5.5 Commit Templates

The commit message dropdown offers:

- **Recent messages:** The last 10 used commit messages
- **Conventional commit templates:** Predefined templates following the Conventional Commits convention:
  - `feat:` , `fix:` , `docs:` , `style:` , `refactor:` , `test:` , `chore:` , etc.

### 5.6 AI Commit Messages

If an AI provider is configured (see AI/MCP Settings):

- The **AI** button in the Commit Dialog automatically generates a commit message based on the staged diff
- The message follows the Conventional Commits convention
- You can edit the suggestion before confirming

---

## 6. Branches

### 6.1 Create a Branch

Methods:

1. **Graph context menu:** Right-click a commit > **Create branch**
2. **Sidebar:** Right-click a branch > **Create branch**
3. **Commit Dialog:** "Create branch" button in the top bar

Options:

- **Name:** Name of the new branch
- **Start point:** Commit or branch to start from (default: selected commit)
- **Checkout:** Option to immediately checkout the new branch

### 6.2 Checkout a Branch

The Checkout Dialog (accessible from the graph or sidebar) provides:

| Option | Description |
|---|---|
| Local branch | Select from local branches |
| Remote branch | Automatically creates a local tracking branch |
| Commit hash | Checkout in detached HEAD mode |
| Local changes handling | None / Merge / Stash / Reset |

### 6.3 Rename and Delete Branches

- **Rename:** Right-click a branch > Rename > Enter the new name
- **Delete local:** Right-click > Delete branch (with force option for unmerged branches)
- **Delete remote:** Right-click > Delete remote branch (requires confirmation)

### 6.4 Delete Remote Branches

Available from both the graph (right-click) and the sidebar. For bulk deletion of obsolete branches, use **Tools > Stale remote branches** (see section 16.3).

### 6.5 Set Upstream Branch

Right-click a branch > **Set upstream** to configure the remote tracking branch. A dialog allows you to select the remote and upstream branch name.

---

## 7. Merge

### 7.1 Merge Dialog

Accessible from:

- Toolbar: **Merge** button
- Graph: Right-click > **Merge into current branch**
- Sidebar: Right-click a branch > **Merge**

### 7.2 Merge Options

| Option | Description |
|---|---|
| Branch | Branch to merge into the current branch |
| Fast-forward | Allow fast-forward when possible |
| No fast-forward | Force creation of a merge commit |
| Squash | Combine all commits into a single commit |
| No commit | Perform the merge without creating the commit automatically |
| Allow unrelated histories | Allow merging unrelated histories |
| Add log messages | Add commit messages to the merge message |

| | |
|---|---|
| Custom message | Custom message for the merge commit |

### 7.3 Abort and Continue

If the merge generates conflicts:

- The **Conflict Banner** appears with Resolve / Abort / Continue options
- **Abort:** Completely cancel the merge, restoring the previous state
- **Continue:** After resolving conflicts, proceed with the merge commit

---

## 8. Rebase

### 8.1 Standard Rebase

Accessible from:

- Toolbar: **Rebase** button
- Graph: Right-click > **Rebase current onto**
- Sidebar: Right-click a branch > **Rebase**

Available options:

| Option | Description |
|---|---|
| Onto | Branch or commit to rebase onto |
| Preserve merges | Keep merge commits |
| Autosquash | Automatically reorder fixup! and squash! commits |
| Auto-stash | Automatically save and restore local changes |
| Update refs | Update branch references during the rebase |

### 8.2 Interactive Rebase

Enabling the **Interactive** option in the Rebase Dialog opens the interactive rebase editor:

- **Commit table:** List of commits that will be rebased
- **Available actions for each commit:**

| Action | Key | Color | Description |
|---|---|---|---|
| Pick | P | Green | Keep the commit |
| Reword | R | Blue | Edit only the message |
| Squash | S | Orange | Merge with the previous commit |
| Fixup | F | Yellow | Like squash but discard the message |
| Edit | E | Purple | Stop to modify the commit |
| Drop | D | Red | Delete the commit |

- **Drag and drop:** Reorder commits by dragging them

- **Keyboard shortcuts:** Press the corresponding letter to change the action

### 8.3 Conflict Handling During Rebase

During a rebase with conflicts:

- The banner shows progress (e.g., "Step 3/7")
- Options: **Skip** (skip the commit), **Abort** (cancel the rebase), **Continue** (proceed to the next step)

---

## 9. Cherry-Pick

To apply a specific commit to the current branch:

1. In the graph, right-click the desired commit
2. Select **Cherry-pick**
3. The commit is applied to the current branch

If conflicts arise, the banner will appear with Abort / Continue options.

---

## 10. Revert

To undo the changes introduced by a specific commit:

1. In the graph, right-click the commit you want to revert
2. Select **Revert**
3. A new commit is created that reverses the changes of the selected commit

If the revert results in conflicts, the conflict banner will appear with the standard Abort / Continue options.

---

## 11. Squash

Squash allows you to combine multiple commits into a single commit:

1. In the graph, right-click a commit > **Squash commits**
2. A preview dialog shows the commits that will be squashed
3. Review the combined changes before confirming
4. The selected commits are condensed into one commit

This is useful for cleaning up a series of small, incremental commits into a single meaningful one before merging.

---

## 12. Conflict Resolution

When an operation generates conflicts, GitPress offers three resolution methods:

### 12.1 Built-in 3-Way Editor

The built-in editor shows a three-column view:

- **Ours (left):** The version from the current branch
- **Base (center):** The common ancestor version
- **Theirs (right):** The version from the incoming branch

For each conflicting section, the following buttons are available:

| Button | Description |
|---|---|
| Accept Ours | Use the current branch version |
| Accept Theirs | Use the incoming branch version |
| Accept Both | Keep both versions |
| Custom | Manually edit the resolution |

The file list on the left shows the conflict count and resolution status for each file.

### 12.2 External Merge Tool

GitPress supports external merge tools:

- **KDiff3**
- **Meld**
- **Beyond Compare**
- And others (configurable)

Configuration in **Settings > Merge Tool**: select the tool, specify the executable path, and the argument pattern.

### 12.3 AI-Assisted Resolution

If an AI provider is configured:

- The **"Resolve with AI"** button analyzes the ours/theirs/base versions
- Shows a preview of the suggested resolution with LCS diff
- You can accept or modify the suggestion before applying it

---

## 13. Diff and File Viewing

### 13.1 Diff Viewer

The diff viewer supports two modes:

- **Line-by-line (unified):** Shows changes in a single view with colored lines
- **Side-by-side:** Shows the before and after versions side by side

Features:

- Syntax highlighting
- Synchronized line numbers
- Colors: red for removed lines, green for added lines
- For images: side-by-side viewer with before/after comparison

### 13.2 File History

Shows the commit history for a single file:

1. Right-click a file > **File History**
2. A virtualized list appears with all commits that modified that file
3. Supports `git log --follow` to track renamed files
4. Click a commit to view the diff for that file at that commit

### 13.3 Blame View

Shows line-by-line annotation of a file:

1. Right-click a file > **Blame**
2. For each line: author, commit hash, and date are displayed
3. Useful for understanding who wrote each line and when

### 13.4 Commit Comparison

To compare two commits:

1. In the graph, right-click a commit > **Compare with HEAD** (compare with the current commit)
2. Or select a commit, then right-click another > **Compare with selected commit**
3. The **Commit Compare Dialog** opens with the list of modified files and the diff for each

### 13.5 File Context Menu

In all panels where files appear, right-click offers:

| Item | Description |
|---|---|
| File History | Commit history for this file |
| Blame | Blame view with line-by-line annotations |
| Open | Open the file with the default application |
| Show in Folder | Open the containing folder in the file manager |
| Copy Path | Copy the file path to the clipboard |
| Add to .gitignore | Add the file/pattern to .gitignore |

---

## 14. Tags

### Create a Tag

1. Right-click a commit in the graph > **Create tag**
2. Or right-click in the Tags section of the sidebar > **Create tag**

Options:

| Option | Description |
|---|---|
| Name | Tag name |
| Message | Message for annotated tags (optional) |
| Annotated | If selected, creates an annotated tag with a message |
| Push to remote | Push the tag immediately after creation |

### Delete a Tag

- **Local:** Right-click > **Delete tag**
- **Remote:** In the deletion confirmation, check the **Delete remote tag** option to also delete from the remote

- You can also delete remote tags from the sidebar

---

## 15. Stash

The Stash Dialog offers two modes:

### Creation Mode (Working Directory)

- Shows a list of modified files in a tree
- Options: include untracked, keep index, staged only
- Optional message field to describe the stash

### Management Mode (Stash List)

- List of all saved stashes with descriptions
- For each stash:

| Action | Description |
|---|---|
| Apply | Apply the stash without removing it from the list |
| Pop | Apply the stash and remove it from the list |
| Drop | Delete the stash from the list |
| Inspect | View the files and changes contained |

---

## 16. Remotes

### 16.1 Fetch, Pull, and Push

#### Fetch

- **Fetch:** Download updates from the default remote
- **Fetch All:** Download from all configured remotes
- **Fetch Prune:** Download and remove references to deleted remote branches

#### Pull

- **Pull (merge):** Download and integrate with merge
- **Pull (rebase):** Download and integrate with rebase

#### Push

- Push to the default remote
- Options: force push, set upstream

All these operations display the **Git Operation Log Dialog** with real-time output and the ability to cancel.

### 16.2 Remote Management

From the sidebar or context menu:

- **Add remote:** Add a new remote (name + URL)
- **Remove remote:** Remove a remote
- **Edit URL:** Modify the URL of an existing remote

### 16.3 Stale Remote Branches

Menu: **Tools > Stale remote branches...**

This tool finds remote branches older than a configurable threshold:

1. Set the **age threshold** (e.g., 30, 60, 90 days)
2. Start the search
3. Branches that haven't received commits beyond the threshold are shown
4. Select the branches to delete
5. Confirm the bulk deletion

---

## 17. Reset

The Reset Dialog allows you to move HEAD to a specific commit:

| Mode | Description |
|---|---|
| **Soft** | Moves only HEAD; staging area and working tree remain unchanged |
| **Mixed** | Moves HEAD and resets the staging area; working tree remains unchanged |
| **Hard** | Moves HEAD, resets staging area and working tree. **Warning: changes will be lost!** |

Accessible from the graph: right-click a commit > **Reset current branch to this commit**.

The modes are color-coded in the dialog to highlight the risk level.

---

## 18. Reflog

The Reflog provides a safety net by showing every change made to HEAD:

1. Access via the sidebar or graph context menu
2. Shows a chronological list of all ref updates (commits, checkouts, rebases, resets, etc.)
3. Each entry displays the action, the resulting commit hash, and a description
4. You can navigate to any reflog entry to inspect or restore a previous state

The reflog is essential for recovering from mistakes like accidental resets or deleted branches.

---

## 19. Bisect

Git Bisect helps you find the commit that introduced a bug using binary search:

1. Open the Bisect Dialog
2. **Start a bisect session** by specifying a known **bad** commit (where the bug exists) and a known **good** commit (where the bug doesn't exist)
3. Git automatically checks out a commit halfway between the two
4. Test the code, then mark the commit as:
    - **Good** — the bug is not present
    - **Bad** — the bug is present
    - **Skip** — cannot determine (e.g., build failure)
5. Repeat until Git identifies the exact commit that introduced the bug

6. **Reset** to end the bisect session

The Bisect Dialog shows the current progress and provides a log of all bisect steps.

---

## 20. Worktrees

Worktrees allow you to work on multiple branches simultaneously without switching:

1. **List worktrees:** View all existing worktrees for the repository
2. **Add worktree:** Create a new worktree at a specified path, optionally creating a new branch
3. **Remove worktree:** Delete a worktree (with force option for dirty worktrees)

Each worktree is an independent working directory linked to the same repository, allowing parallel development on different branches.

---

## 21. Submodules

From the **Submodules** section of the sidebar:

| Action | Description |
|---|---|
| Add | Add a new submodule (URL + optional path) |
| Initialize | Initialize submodules that are not yet initialized |
| Update | Update submodules to the latest registered commit |
| Sync | Synchronize submodule remote URLs |
| Deinitialize | Unregister a submodule |
| Status | View the current status of all submodules |

---

## 22. Git LFS (Large File Storage)

Git LFS management is accessible via a dedicated dialog:

| Action | Description |
|---|---|
| Install LFS | Install Git LFS hooks in the repository |
| Status | View LFS status and tracked files |
| Track | Add file patterns to be tracked by LFS (e.g., `*.psd`, `*.zip`) |
| Untrack | Remove file patterns from LFS tracking |
| Storage info | View LFS storage usage information |

LFS is useful for managing large binary files (images, videos, compiled assets) without bloating the repository history.

---

## 23. Patches

Patches allow you to share changes outside of the normal push/pull workflow:

### Create a Patch

1. Select one or more commits in the graph
2. Right-click > **Create patch** (or use the Patch Dialog)
3. A `.patch` file is generated using `git format-patch`

### Apply a Patch

1. Open the Patch Dialog
2. Select a `.patch` file to apply
3. Preview the patch content before applying
4. Apply the patch to the current branch

---

## 24. Archive and Export

Export a snapshot of the repository at any ref as an archive:

1. Open the Archive Dialog
2. Select the ref (branch, tag, or commit) to export
3. Choose the format: **ZIP** or **tar.gz**
4. Select the destination path
5. The archive is generated from the selected ref

This is useful for creating release packages or sharing code snapshots without Git history.

---

## 25. Git Notes

Git Notes allow you to attach additional information to commits without modifying the commit itself:

| Action | Description |
|---|---|
| View notes | Display notes attached to a commit |
| Add note | Attach a note to a specific commit |
| Remove note | Remove a note from a commit |

Notes are useful for adding review comments, deployment metadata, or other annotations to commits after the fact.

---

## 26. Pull Request Integration

GitPress integrates with popular Git hosting providers for pull request management:

### Supported Providers

- **GitHub**
- **GitLab**
- **Gitea**
- **Bitbucket**

The provider is automatically detected from the remote URL.

**Features**

| Feature | Description |
|---|---|
| List PRs | View all open pull requests for the repository |
| View PR details | See the description, status, and changes of a PR |
| Create PR | Create a new pull request from the current branch |
| AI PR description | Generate a PR description automatically using AI (if configured) |

---

## 27. Changelog

To generate a changelog between two points in history:

1. Right-click a commit in the graph > **Generate changelog**
2. Select the start and end tag/commit
3. The changelog is automatically generated, grouped by Conventional Commit type:
   - **Features** ( `feat:` )
   - **Bug Fixes** ( `fix:` )
   - **Documentation** ( `docs:` )
   - **Styles** ( `style:` )
   - **Refactoring** ( `refactor:` )
   - **Tests** ( `test:` )
   - **Chores** ( `chore:` )

The dialog can also be opened as a separate window.

---

## 28. Integrated Console

GitPress includes a built-in terminal (similar to the Console in Git Extensions):

- **Shell:** Automatically detects the system shell (bash/zsh on Linux, cmd/PowerShell on Windows)
- **Font:** Cascadia Code (fallback: Consolas, Courier New)
- **Colors:** Synchronized with the application theme
- **Scrollback:** 5000 lines of history
- **Resizing:** Automatically adapts to the panel size

The terminal opens automatically in the current repository directory.

---

## 29. Command Log

The **Command Log** panel shows all Git commands executed by the application:

- Timestamp for each command
- Full command with arguments
- Maximum of 200 entries in the history
- **Clear** button to empty the log

Useful for debugging and understanding which Git operations the interface executes.

### Git Operation Log Dialog

For operations that may take time (push, pull, fetch, merge, rebase, cherry-pick):

- A modal dialog opens automatically with real-time output
- Shows stdout and stderr from the Git process
- **Cancel** button to interrupt the operation
- **Auto-close on success** checkbox to automatically close after 1.5 seconds on success
- On error, the message is displayed in a red box

---

## 30. Statistics

### 30.1 Author Statistics

The **Author Statistics** panel shows a contributor leaderboard:

#### Leaderboard

- Sortable by: commits, additions, deletions, files touched
- Time filter: All Time, Last Month, Last Week
- Gravatar avatar for each author

#### Author Detail (click to expand)

- **Sparkline:** Line chart of activity over the last 52 weeks
- **Heatmap:** GitHub-style heat map of daily activity
- **Top Files:** Files the author has worked on most
- **Streaks:** Consecutive days with commits

Statistics update automatically after an auto-fetch.

### 30.2 Codebase Statistics

The **Codebase Statistics** panel analyzes the repository's code:

- **LOC by language:** Bar chart with 70+ supported file extensions
- **LOC by type:** Breakdown into source, test, config, styles, docs, CI/CD, other
- **Test Ratio:** Bar showing the ratio of test code to source code

---

## 31. Git Accounts

GitPress supports managing multiple Git identities:

### Account Configuration

In **Settings** > **Accounts**:

1. Add one or more accounts with:
    - Name
    - Email
    - Signing key (optional)
    - SSH key path (optional)
2. Set a **default account**

### Per-Repository Assignment

- Each repository can have a specific account assigned
- The account selector in the toolbar shows the active account
- Changing the account automatically updates `user.name` and `user.email` in the local repo config

### SSH Import

GitPress can parse the `~/.ssh/config` file to import existing SSH configurations.

---

## 32. Settings

Open with: `Ctrl+,` or **Tools > Settings...**

### 32.1 General

| Setting | Description |
|---|---|
| Theme | Dark (Catppuccin Mocha) or Light (Catppuccin Latte) |
| Language | Application language (see Internationalization) |
| Auto-fetch | Enable/disable periodic automatic fetching |
| Fetch interval | Interval in seconds between automatic fetches |
| Prune on auto-fetch | Remove obsolete remote references during auto-fetch |

### 32.2 Accounts

Multiple Git account management (see section 31).

### 32.3 Git Config

Direct editing of Git configuration keys:

- `user.name` and `user.email`
- Core settings
- Defaults for merge and pull
- Configuration at local (repo) or global level

### 32.4 Fetch

- Default remote
- Prune settings

### 32.5 Commit

| Setting | Description |
|---|---|
| Default template | Default template for commit messages |
| Sign commits | Sign commits with GPG/SSH |
| Default message | Default message for new commits |

### 32.6 Diff and Graph

| Setting | Description |
|---|---|
| Context lines | Number of context lines in the diff |
| Side-by-side | Preference for side-by-side view |
| Max initial load | Maximum number of commits initially loaded in the graph |
| Show remote branches | Show/hide remote branches in the graph |

### 32.7 Merge Tool

| Setting | Description |
|---|---|
| Tool name | Name of the tool (KDiff3, Meld, Beyond Compare, etc.) |
| Tool path | Path to the executable |
| Arguments pattern | Argument pattern to pass to the tool |

### 32.8 Advanced

| Setting | Description |
|---|---|
| Max concurrent processes | Maximum number of parallel Git processes |
| Git binary path | Custom path to the Git executable (with Browse button) |

### 32.9 AI / MCP

| Setting | Description |
|---|---|
| Provider | none / Anthropic / OpenAI / Google Gemini / Custom MCP |
| API key | API key for the selected provider |
| Model | Specific model to use |
| Base URL | Custom URL for the API endpoint |
| MCP Server | Enable/disable the built-in MCP server |

---

## 33. AI and MCP Integration

### 33.1 Supported AI Providers

| Provider | Description |
|---|---|
| **Anthropic** | Claude (claude-3-opus, claude-3-sonnet, etc.) |
| **OpenAI** | GPT-4, GPT-3.5, etc. |
| **Google Gemini** | Gemini Pro, etc. |
| **Custom MCP** | Any server compatible with the MCP protocol |

### 33.2 MCP Server

GitPress includes a **built-in MCP server** that exposes 50+ Git operations as tools for external AI assistants:

- Read operations: status, log, diff, blame, branch list, etc.
- Write operations: commit, checkout, merge, push, etc.
- Communication via standard MCP stdio protocol

To enable it: **Settings > AI / MCP > MCP Server > Enabled**

### 33.3 AI Features

When an AI provider is configured, the following features become available:

| Feature | Where | Description |
|---|---|---|
| **Commit message** | Commit Dialog | Generates a conventional commit message from the diff |
| **Conflict resolution** | Merge Conflict Dialog | Suggests conflict resolution with diff preview |
| **PR description** | Toolbar/Menu | Generates a Pull Request description from commits |
| **Code review** | Graph (right-click) | Code review for a specific commit |

---

## 34. Automatic Updates

GitPress supports automatic updates via GitHub Releases:

1. **Manual check:** Menu **Help > Check for updates...**
2. **Automatic notification:** The application periodically checks for new versions
3. When available, a notification is shown with the option to download and install

The update is downloaded in the background and installed on restart.

### Antivirus Scan

Every release is automatically scanned with VirusTotal, which checks binaries against approximately 70 antivirus engines. Scan results with direct links to full reports are included in each GitHub release. If any file receives 3 or more malicious detections, the release is blocked until investigated.

---

## 35. Keyboard Shortcuts

### Global

| Shortcut | Action |
|---|---|
| `Ctrl+O` | Open repository |
| `Ctrl+N` | Create new repository (when no repo is open) |
| `Ctrl+K` | Open Commit Dialog |
| `Ctrl+Q` | Exit the application |
| `Ctrl+,` | Open Settings |

| | |
|---|---|
| `Ctrl+G` | Open Git bash |
| `F5` | Refresh |

### Commit Graph

| Shortcut | Action |
|---|---|
| `Ctrl+F` | Show/hide the search bar |

### Commit Dialog

| Shortcut | Action |
|---|---|
| `Ctrl+Enter` | Confirm the commit |

### Interactive Rebase

| Shortcut | Action |
|---|---|
| `P` | Pick (keep commit) |
| `R` | Reword (edit message) |
| `S` | Squash (merge) |
| `F` | Fixup (merge without message) |
| `E` | Edit (modify commit) |
| `D` | Drop (delete commit) |

### General

| Shortcut | Action |
|---|---|
| `Escape` | Close context menus and dialogs |
| `Enter` | Confirm in input fields |

---

## 36. Themes

GitPress offers two themes based on the **Catppuccin** palette:

### Dark Theme (Catppuccin Mocha)

- Dark background with light text
- Vibrant accent colors
- High readability with optimized contrast

### Light Theme (Catppuccin Latte)

- Light background with dark text
- Accent colors adapted for the light background

- High contrast for readability

Change the theme from: **Settings > General > Theme**

The theme is saved and applied immediately without restarting.

---

## 37. Customizable Layout

The interface uses the **dockview** system for dockable panels:

- **Drag & drop:** Drag panels to reposition them
- **Tabs:** Group multiple panels in the same area with tabs
- **Resizing:** Drag borders to resize panels
- **Persistence:** The layout is saved per repository and automatically restored

### Available Panels

| Panel | Description |
|---|---|
| Sidebar | Browser for branches, remotes, tags, stash, submodules |
| Commit Graph | Commit graph with colored lines |
| Commit Info | Details of the selected commit |
| Diff / Files | Diff viewer and file tree of the commit |
| Command Log | History of executed Git commands |
| Console | Integrated terminal |
| Author Statistics | Statistics by author |
| Codebase Statistics | Source code statistics |

### Reset Layout

If the layout becomes problematic: **Dashboard > Reset layout** restores the default arrangement.

---

## 38. Internationalization

GitPress supports multiple languages via the internationalization (i18n) system:

- Change the language from **Settings > General > Language**
- All UI elements, menus, dialogs, and messages are translated
- The language setting is applied immediately

---

## Appendix: Troubleshooting

### Git Not Found

If Git is not in the system PATH, configure the path manually in **Settings > Advanced > Git binary path**.

### Performance with Large Repositories

- Reduce the number of initially loaded commits in **Settings > Diff & Graph > Max initial load**
- Use branch filters to limit the view
- Disable "Show remote branches" if not needed

### Auto-fetch Not Working

Verify that **Settings > General > Auto-fetch** is enabled and the interval is configured correctly.

### "Dubious Ownership" Error

Git may refuse to open a repository owned by a different user. GitPress automatically handles this by passing `-c safe.directory=<path>` to every git command, so no manual configuration is needed.

### Conflicts Not Detected

Make sure Git is updated (2.30+). Press `F5` to refresh the repository status.

---

*GitPress is open-source software released under the MIT license. Repository: [https://github.com/CODEPRESS-SRL/git-press](https://github.com/CODEPRESS-SRL/git-press) Report bugs: [https://github.com/CODEPRESS-SRL/git-press/issues](https://github.com/CODEPRESS-SRL/git-press/issues)*